

motorBench 2.55.0 User's Guide

READY

 25 Feb 2026

Table of contents:

- [Introduction](#)(see page 2)
- [Recommended reading](#)(see page 2)
- [Installation](#)(see page 2)
- [Hardware setup](#)(see page 3)
 - [MCLV-2](#)(see page 3)
 - [dsPIC33CK LVMC](#)(see page 7)
 - [MCLV-48V-300W](#)(see page 8)
 - [MCHV-230VAC-1.5kW](#)(see page 10)
 - [Using an Encoder](#)(see page 13)
 - [Encoder signals and wiring requirements](#)(see page 13)
 - [Number of lines for the encoder](#)(see page 14)
 - [Using the motor's index pulse](#)(see page 15)
 - [Ensuring correct phase sequence between the motor and encoder](#)(see page 15)
 - [Real-time debugging tool](#)(see page 16)
 - [Oscilloscope](#)(see page 16)
- [Operating instructions](#)(see page 16)
 - [MCC configuration for motorBench projects](#)(see page 16)
 - [MCC Classic](#)(see page 17)
 - [MCC Melody](#)(see page 17)
 - [Using MCC Classic](#)(see page 17)
 - [Using MCC Melody](#)(see page 17)
 - [Configuration](#)(see page 18)
 - [Board configuration](#)(see page 19)
 - [Default configuration](#)(see page 19)
 - [Supported development boards](#)(see page 19)
 - [Support for custom boards](#)(see page 20)
 - [Creating and using a custom board definition file](#)(see page 20)
 - [Making changes to custom board definition in an existing project](#)(see page 20)
 - [Comparing board definition](#)(see page 21)
 - [System parameters](#)(see page 21)
 - [Motor import and export](#)(see page 22)
 - [Importing a motor](#)(see page 22)
 - [Exporting a motor](#)(see page 22)
 - [Working with custom motors](#)(see page 23)
 - [Motor parameter measurement](#)(see page 23)
 - [Operating instructions](#)(see page 23)
 - [Selective measurements feature](#)(see page 24)
 - [Inductance calculator widget](#)(see page 25)
 - [Common types of motor parameter measurement errors and reasons they can occur](#)(see page 26)
 - [Autotuning](#)(see page 27)
 - [Managing errors in autotuning](#)(see page 28)
 - [How to identify autotuning errors](#)(see page 28)
 - [Common types of autotuning errors and reasons they can occur](#)(see page 29)
 - [How to report specific failure details to Microchip for assistance](#)(see page 30)
 - [Customize](#)(see page 30)

- Position and velocity estimator selection(see page 30)
- Advanced parameters(see page 31)
- Normalized parameters(see page 31)
- Advice(see page 31)
- Additional information(see page 32)
- Generate code(see page 32)
 - Managing errors in code generation(see page 32)
 - How to identify code generation errors(see page 32)
 - Common types of code generation errors and reasons they can occur(see page 33)
 - How to report specific failure details to Microchip for assistance(see page 35)
- Building code(see page 35)
- Running the Motor Control Application Framework(see page 36)
 - MCLV-2(see page 36)
 - dsPIC33CK LVMC and MCLV-48V-300W(see page 36)
 - MCHV-230VAC-1.5kW(see page 36)
- Real-Time Diagnostics(see page 36)
- Troubleshooting issues with some motors(see page 37)
- Appendix(see page 37)
 - Motor parameter measurement - Calibration load specifications(see page 37)
 - Loading Custom motorBench Firmware Packages in a Project for Advanced Users(see page 38)

Introduction

This document describes how to set up hardware, configure, and operate motorBench® Development Suite as well as aid in troubleshooting issues with motors.

For more information on the code that motorBench generates from the Motor Control Application Framework (MCAF), refer to the [MCAF User's Guide](#)¹.

Recommended reading

This user's guide covers motorBench operation only. Other useful documents are listed below:

- MPLAB® Code Configurator (MCC) User's Guide
 - refer to <https://www.microchip.com/en-us/tools-resources/configure/mplab-code-configurator>
- motorBench® v2.55.0 Release Notes
 - refer to <https://github.com/microchip-pic-avr-solutions/motorbench-release-collateral>
- [MCAF User's Guide](#)²

Installation

The following instructions outline the installation procedure for motorBench® Development Suite:

1. Verify that your internet connection is stable.
2. Open MPLAB X IDE
3. Open the Content Manager from **Tools** → **MCC Content Manager**.
4. Navigate to motorBench® Development Suite under **Libraries** section. Select "2.55.0" from the drop-down menu and then click **Apply**.
5. MCC will automatically attempt to download a copy of motorBench® Development Suite v2.55.0.
6. In the drop-down menu for motorBench versions, verify that v2.55.0 shows up as "local".

¹ <https://microchiptech.github.io/mcaf-doc/latest/>

² <https://microchiptech.github.io/mcaf-doc/latest/>

If the Content Manager fails to download and install v2.55.0 automatically, then follow the manual procedure below:

1. Download a copy of the motorBench® Development Suite v2.55.0 library file from the [MPLAB® Code Configurator website](#)³
 - a. Scroll to the MCC Content Types section and click the **View MCC Classic** button under MCC Classic
 - b. Scroll to MPLAB Code Configurator (MCC) Classic Libraries and locate **motorBench® Development Suite v2.55.0** and click to download.
2. Open MPLAB X IDE
3. Go to **Tools** → **Options** → **Plugins** and then use the **Install library** button to manually install v2.55.0.

Hardware setup

The following sections outline the hardware setup for [MCLV-2](#)⁴, [dsPIC33CK LVMC](#)⁵, [MCLV-48V-300W](#)⁶, and [MCHV-230VAC-1.5kW](#)⁷.

-  motorBench v2.50.0 is the last version of motorBench to include full support for dsPICDEM MCHV-2 Development Board ([DM330023-2](#)⁸) and dsPICDEM MCHV-3 Development Board ([DM330023-3](#)⁹). motorBench v2.55.0 has partial support for the stock versions of these two boards:
-  Motor parameter measurement is not supported
 -  Programming via ICD connection is not supported; it requires MPLAB X IDE v.6.20 or earlier
 -  All other features of motorBench are supported, including generating code and using MPLAB X to compile and link the code (MPLAB Integrated Programming Environment (IPE) 6.20 may be used to program a device from the resulting .hex file.)

Full support for these two boards can be restored by replacing the onboard isolated programmer on these two boards with the new High Voltage Motor Control Isolated Debugger Card ([DV43T08A](#)¹⁰). For AC mains-powered development work, please use MCS MCHV-230VAC-1.5kW Motor Control High-Voltage Development Board ([EV78U65A](#)¹¹).

MCLV-2

Hardware setup: MCLV-2

This section provides detailed steps that will help you set up your hardware to work with the motorBench® Development Suite:

1. The MCLV-2 board comes pre-installed out of the box with dsPIC33EP256MC506 Internal Op Amp Motor Control PIM (MA330031). This PIM may be replaced with any of the supported PIMs listed in the motorBench v2.55.0 Release Notes.

³ <https://www.microchip.com/en-us/tools-resources/configure/mplab-code-configurator>

⁴ <https://www.microchip.com/en-us/development-tool/DM330021-2>

⁵ <https://www.microchip.com/en-us/development-tool/DM330031>

⁶ <https://www.microchip.com/en-us/development-tool/EV18H47A>

⁷ <https://www.microchip.com/en-us/development-tool/ev78u65a>

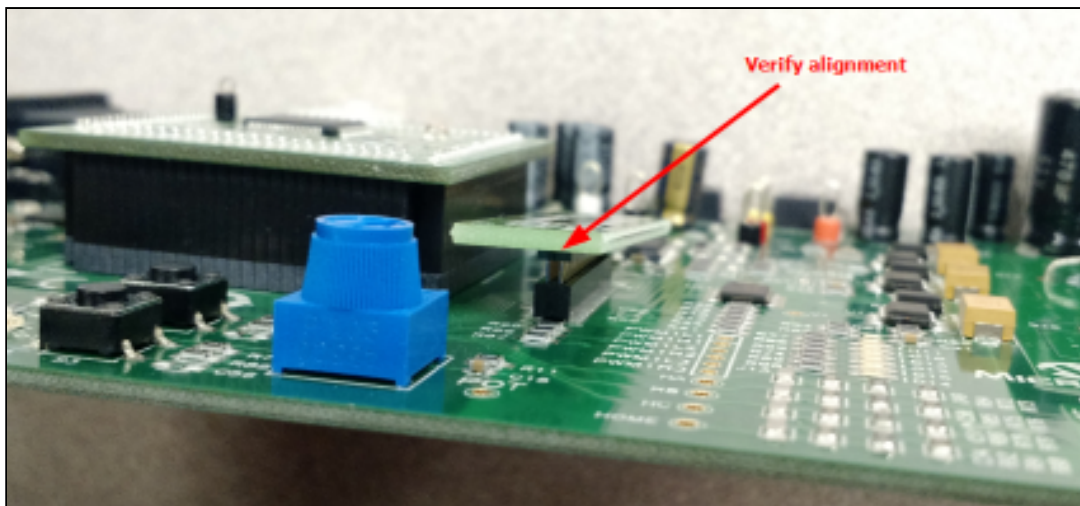
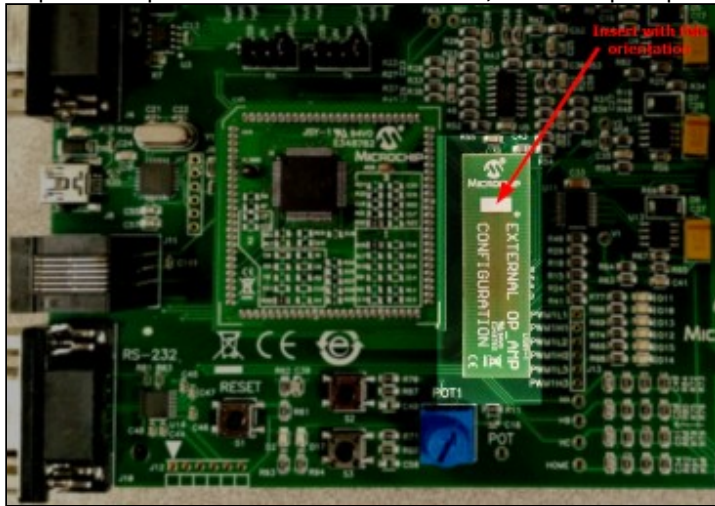
⁸ <https://www.microchip.com/DM330023-2>

⁹ <https://www.microchip.com/DM330023-3>

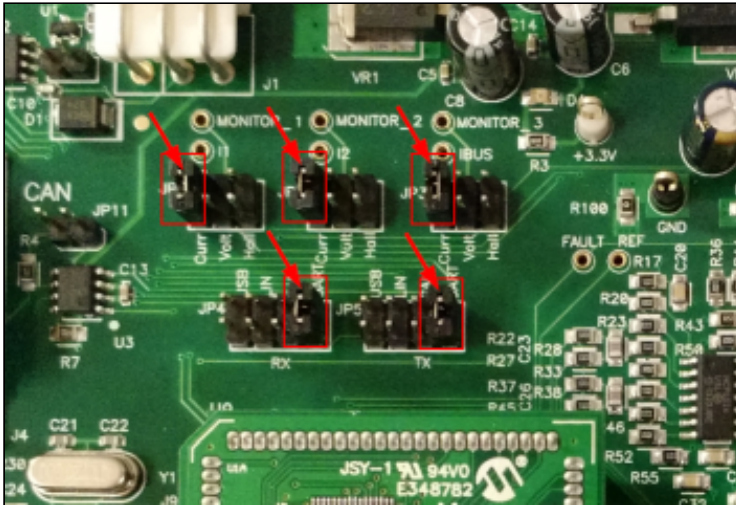
¹⁰ <https://www.microchip.com/en-us/development-tool/dv43t08a>

¹¹ <https://www.microchip.com/ev78u65a>

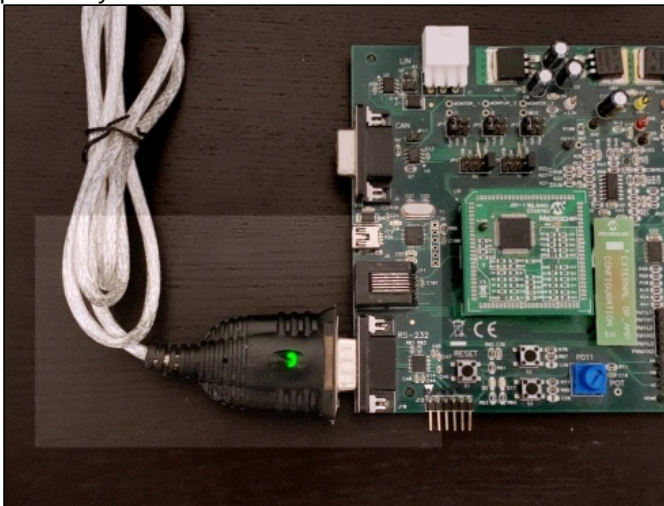
2. If using the dsPIC33EP256MC506 External Op Amp PIM, make sure that the PIM is populated with a silicon mask rev-A8 or later. To verify this, read out the device revision from MPLAB X and verify that the Device ID revision is equal to or greater than 0x4008 .
3. Ensure the Op Amp Configuration matrix board installed in J14 matches the installed PIM. External Op Amp PIMs require the External matrix board; Internal Op Amp PIMs require the Internal matrix board.



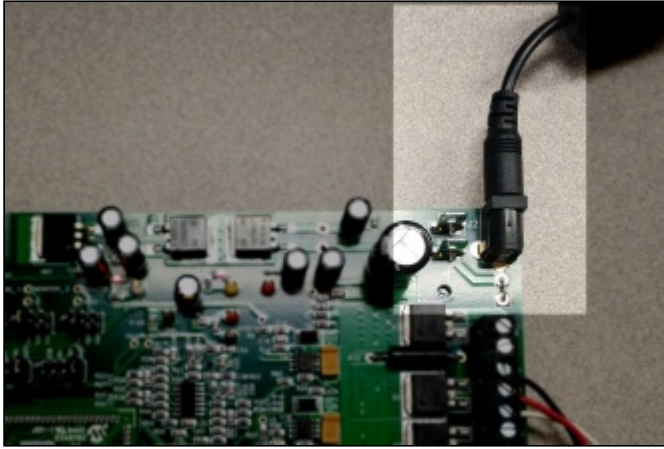
4. Update the jumpers JP1-JP2-JP3 to Curr position and JP4-JP5 to UART position as shown below:



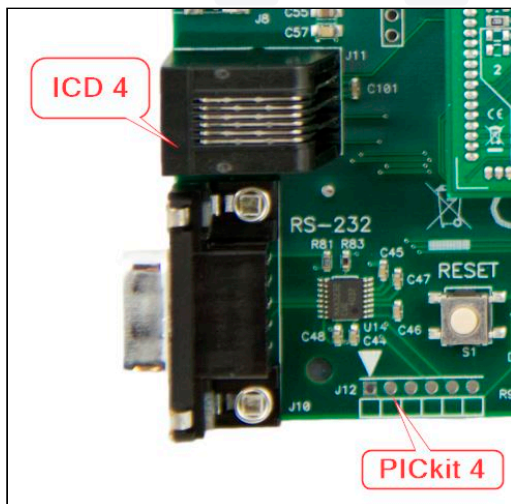
5. Connect the USB-to-logic-level-UART converter cable into J10 of the MCLV-2 board and one of the USB ports on your PC:



6. Plug in the 24V power supply barrel connector into **J2** of the MCLV-2 board and then plug the power supply into the mains outlet:

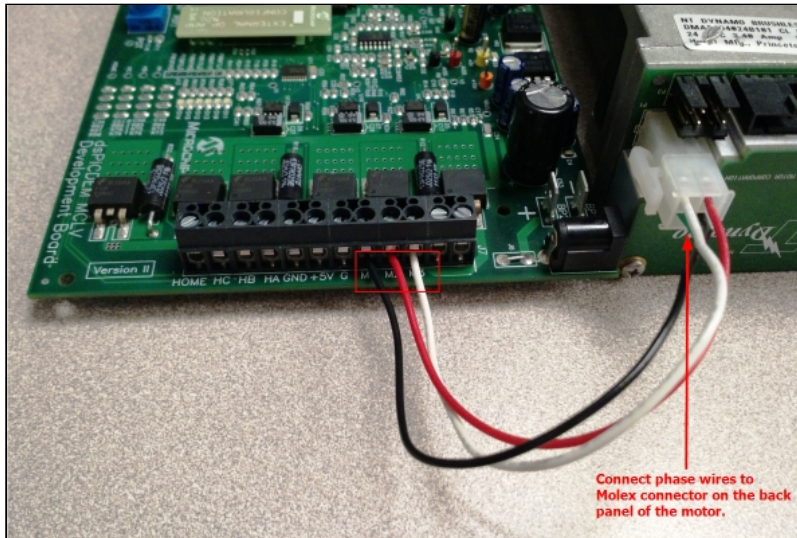


7. Connect PICkit4 or ICD4 to one of the USB ports on PC
8. Connect PICkit4 to **J12** or ICD4 to **J11** of the MCLV-2 board.

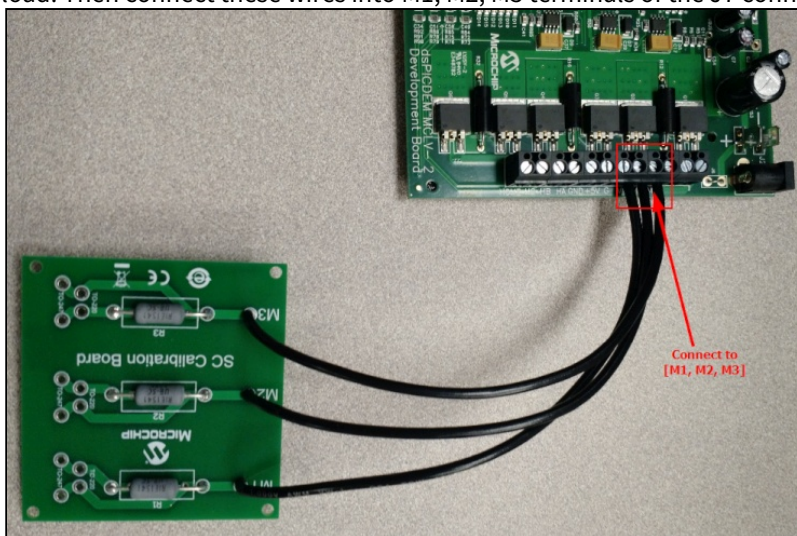


9. Connect the [black, red, white] phase wires of the 24V BLDC motor to [M1, M2, M3] terminals of **J7** connector on the MCLV-2 board. You may leave the green color phase wire unconnected or connect it to the **G** terminal of **J7** connector on the MCLV-2 board. Then, plug in the

other end with a Molex connector to the 24V BLDC motor.



10. Keep the motor on a stable surface and use a clamp (if available) to secure the motor from jumping around. (If using a metal C-clamp, make sure there is a thin shim of rubber, cloth, wood, or other mechanically-compliant material between the clamp and the motor, to avoid deforming the motor housing.) Alternatively, you can also place the motor on a rubber mat. Also, do not disturb the motor or hold its shaft while motor parameter measurement is running.
11. While running the Board Calibration within motor parameter measurement feature in motorBench® Development Suite, you will need to use a calibration load of three equal-value resistors connected in place of the 24V BLDC motor. Refer to the [Appendix section](#)(see page 37) of this document for more information on the calibration load. Once you have the calibration load on hand, you can start by stripping off the insulation on the wires coming from the M1, M2 and M3 terminals of the Calibration load. Then connect these wires into M1, M2, M3 terminals of the J7 connector on the MCLV-2 board.

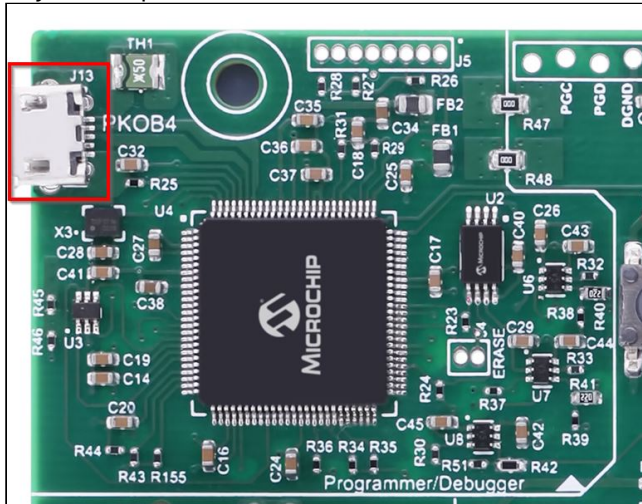


dsPIC33CK LVMC

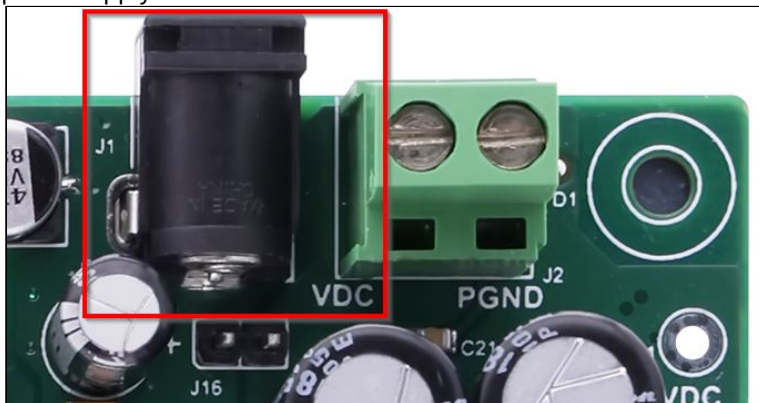
Hardware setup: dsPIC33CK LVMC

This section provides detailed steps that will help you set up your hardware to work with the motorBench® Development Suite:

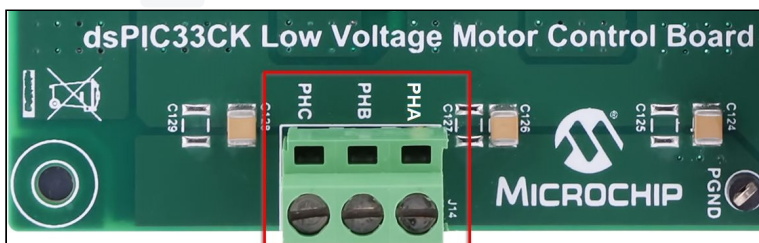
1. Connect the USB Micro-B cable between J13 of the dsPIC33CK LVMC board and one of the USB ports on your computer:



2. Plug in the 24V power supply barrel connector into J1 of the dsPIC33CK LVMC board and then plug the power supply into the mains outlet:



3. Connect the [black, red, white] phase wires of the 24V BLDC motor to [PHA, PHB, PHC] terminals of J14 connector on the dsPIC33CK LVMC board.

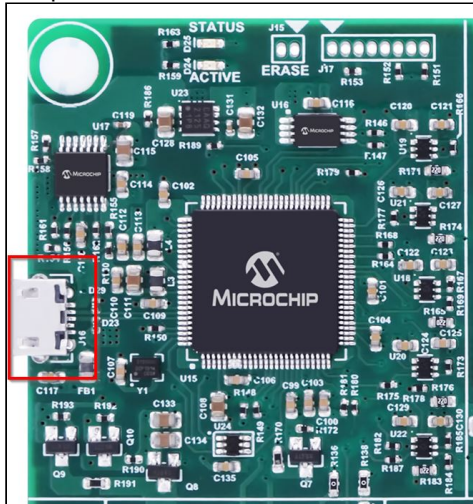


MCLV-48V-300W

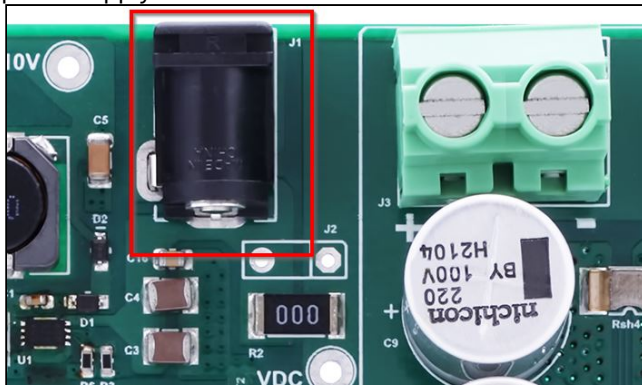
Hardware setup: MCLV-48V-300W

This section provides detailed steps that will help you set up your hardware to work with the motorBench® Development Suite:

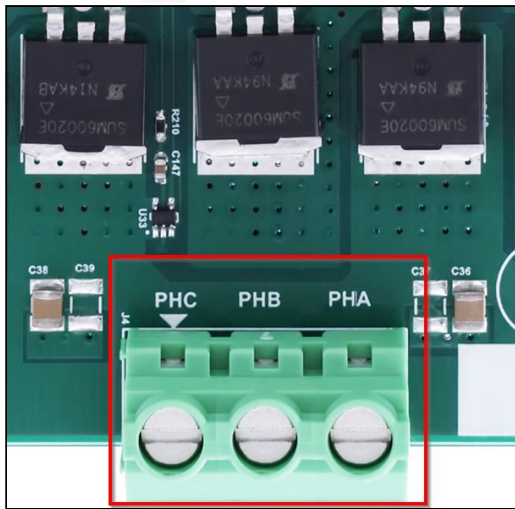
1. The MCLV-48V-300W board comes with the dsPIC33CK256MP508 Internal OpAmp Motor Control DIM (EV62P66A); install the DIM in **J8** on the MCLV-48V-300W board. This DIM may be replaced with any of the supported DIMs (see motorBench v2.55.0 Release Notes for additional information).
2. Connect the USB Micro-B cable to **J16** of the MCLV-48V-300W board and one of the USB ports on your computer:



3. Plug in the 24V power supply barrel connector into **J1** of the MCLV-48V-300W board and then plug the power supply into the mains outlet:



4. Connect the [black, red, white] phase wires of the 24V BLDC motor to [PHA, PHB, PHC] terminals of J4 connector on the MCLV-48V-300W board.



MCHV-230VAC-1.5kW

Hardware setup: MCHV-230VAC-1.5kW



WARNING – This system must be earthed (grounded) at all times.

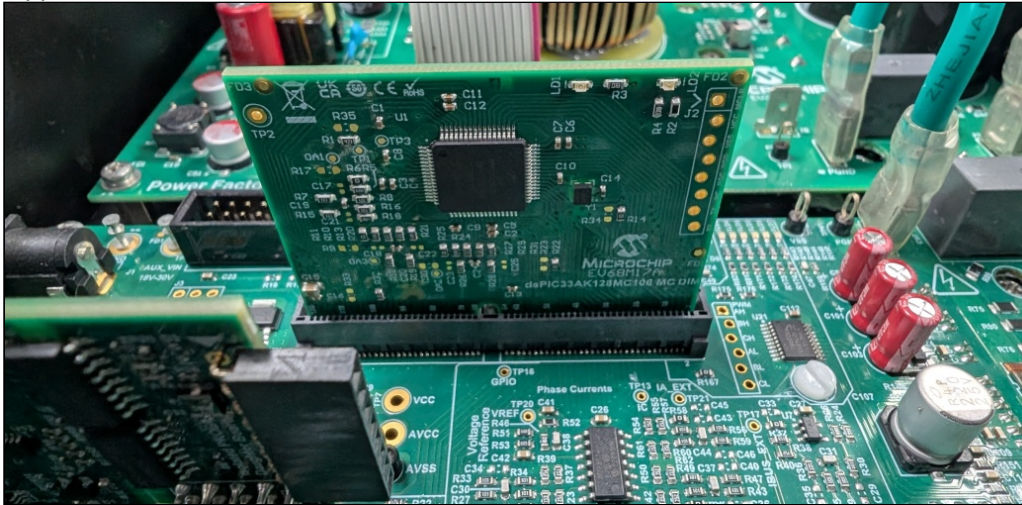
WARNING – The output terminals are NOT isolated from the incoming AC mains supply and may be at up to 410V with respect to ground, regardless of the input mains supply voltage applied. These terminals are live during operation AND for five minutes after disconnection from the supply. Do not attempt to access the terminals or remove the cover during this time.

WARNING – The unit may obtain power through the output terminals if these are connected to a rotating motor acting as a generator. If this is the case, then the previous warning also applies (i.e., the output terminals are live when connected to the generator and for five minutes after the generator has been stopped). Note that this case can arise even when the unit has been disconnected from the incoming AC mains supply.

This section provides detailed steps that will help you set up your hardware to work with the motorBench® Development Suite:

1. The MCHV-230VAC-1.5kW board comes with the dsPIC33AK128MC106 Internal OpAmp Motor Control DIM (EV68M17A). Remove the thumb screw and open the top cover of the enclosure and install the DIM into the DIM Interface connector J2 on the development board. This DIM may be replaced with any of the

supported DIMs (see motorBench v2.55.0 Release Notes for additional information).



- The development board has an isolated on-board programming tool called the Isolated PKoB4 Daughter Board. To use the isolated on-board programmer, connect a micro-USB cable between the Host PC and the connector J11 (**PROGRAM**) on the development board.



- To establish serial communication with the host PC, connect a micro-USB cable between the host PC and the connector J8 (**USB-UART**) on the development board. This interface provides an isolated USB-UART communication.



Note

Use only **shielded micro-USB** cables intended for data transfer

4. Connect the [black, red, white] phase wires of the Leadshine 400W 220VAC Servo Motor (AC300025) to the [M1, M2, M3] terminals on the MCHV-230VAC-1.5kW board.



5. Plug in an appropriate AC power supply cable to IEC connector **J1** provided on the PFC board of MCHV-230VAC-1.5kW board and then plug in the AC power supply cable to the mains outlet.



Using an Encoder

If you are using a quadrature encoder, the following points need to be considered:

- Encoder signals and wiring requirements
- Number of lines for the encoder
- Using the encoder index pulse
- Ensuring correct phase sequence between the motor and encoder

Encoder signals and wiring requirements

Verify signals on the wires coming from the encoder, which may be some combination of the following:

- Single-ended (A/B) or differential (A/A-/B/B-) data lines
- Index pulse, usually "Z"
- Power, which may be +3.3V, +5V, or some other voltage, which must be supplied correctly
- GND
- Other signals as defined in the datasheet

Encoders typically have A and B signals, which may be single-ended or differential, an index pulse Z, power, and GND. For dsPIC devices, only single-ended encoder signals may be used, so if the encoder has differential pair outputs, there are a couple of options available. Generally, differential signals on encoders may be connected as single-ended with the A and B connections plus Z, leaving the differential "A-bar" and "B-bar" signals disconnected, or you may use a differential receiver to interface with the QE1 peripheral on the dsPIC.

Below is an example of wires coming from the encoder on the ACT57BLF02 motor. MCAF + dsPIC only needs to interface with the A+, B+, Z+, +5V, and GND wires to work properly.

Color	Signal
Red	+5V

Color	Signal
Black	GND
Blue	A+
Blue-Black	A
Green	B+
Green-Black	B
Yellow	Z+
Yellow-Black	Z
Brown	U+
Brown-Black	U
Gray	V+
Gray-Black	V
White	W+
White-Black	W
Shield	PE

Number of lines for the encoder

Verify the number of lines for the encoder, which may be stated in the datasheet as "counts per revolution", lines, "pulses per revolution"(PPR), or "cycles per revolution" (CPR). The number of lines, PPR, or CPR is equal to counts per revolution $\div 4$. For example: 256 lines = 256 PPR = 256 CPR = 1024 counts per revolution.

If the wrong number of lines is used in MCAF, the motor will lock up in one position.

Using the motor's index pulse

If the number of lines is a power of two, then the index pulse is compatible with MCAF, and if not, MCAF can be configured on the Customize page of motorBench to ignore the index pulse.

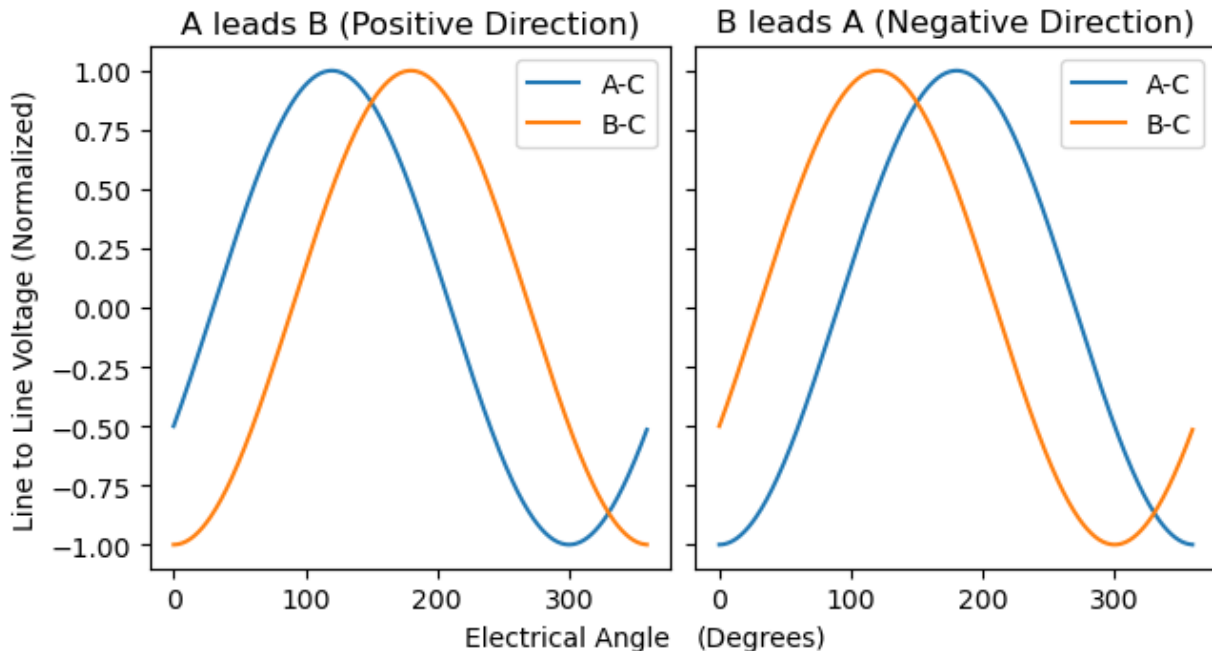
If the number of lines is not a power of two, the encoder can still be used by MCAF without the index pulse, but an encoder alignment routine will be necessary on power-up.

Ensuring correct phase sequence between the motor and encoder

The sequence of the motor phase connections and the proper wiring of the encoder connections is important for the correct motor operation. If the positive motor direction does not match the positive encoder direction, then MCAF will not be able to use the encoder for commutation. In MCAF, "positive" direction is defined as the motor direction when viewing the driveshaft end of the motor when the back EMF phase A leads phase B, but different motors may have clockwise (CW) or counter-clockwise (CCW) defined differently based on "A leads B" logic. Ensure the motor's behavior matches expectation for the direction of shaft rotation when wired with phase A/B/C or U/V/W per the datasheet. The direction of a motor can then be changed by swapping any two phase wires.

To empirically test which direction is "positive", you can measure line-to-line voltages of A and B phases with respect to C phase. To do so, connect oscilloscope probes to the A- and B-phase wires with ground references connected to the C-phase wire while manually spinning the driveshaft of the motor. If the A phase leads the B phase, then the direction the motor is being turned in is the "positive" direction, and if the B phase leads the A phase, then the direction the motor is being turned in is the "negative" direction (as shown below).

Motor Direction vs Leading Phase of Line to Line Back EMF Measurement



The encoder datasheet should define the behavior, CW or CCW, of the encoder when A leads B or B leads A, or there may be a way to set up the encoder to select which direction is "A leads B". This should be verified to match the motor in MCAF by observing the behavior of the encoder's signals while manually turning the motor's driveshaft using either of the following methods. If the encoder and motor positive directions do not match, either the motor

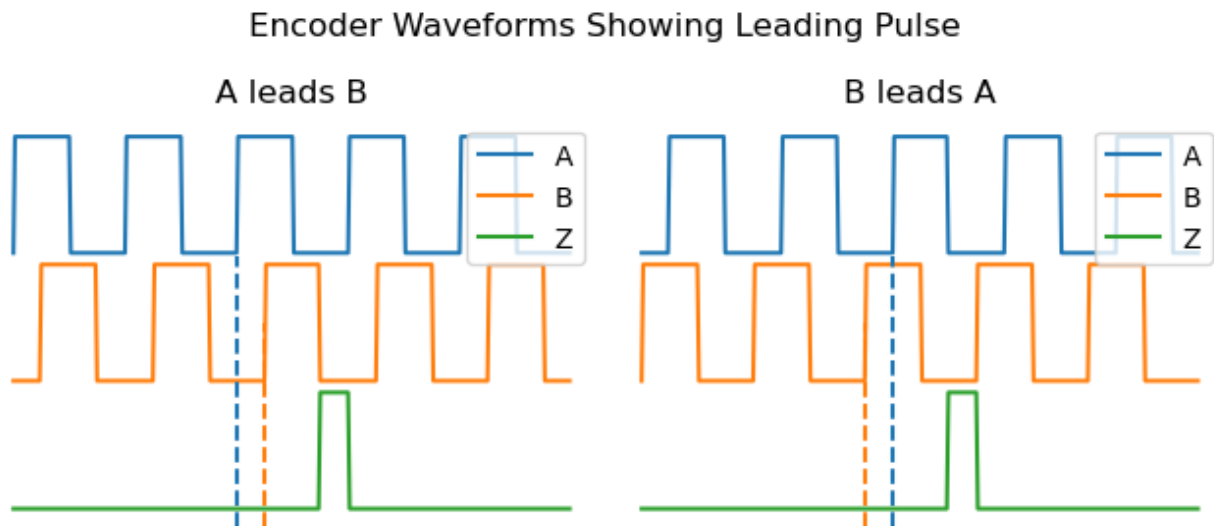
direction can be reversed by swapping any two phase wires, as described above, or the encoder can be reversed by swapping the A and B data wires.

Real-time debugging tool

Use a real-time debugging tool (such as X2Cscope) to observe the angle in MCAF versus the commutation angle. To do so, manually turn the motor's driveshaft in the positive direction and ensure that the encoder angle is also increasing.

Oscilloscope

Use an oscilloscope to observe the encoder signals directly while manually turning the motor's driveshaft. Shown below is an example of possible output waveforms that can be expected by an encoder when observing the data lines directly, refer to the encoder datasheet for more information on the specific encoder used.



i Different encoders may have slight variations to these waveforms. Refer to your encoder's datasheet to see the exact waveforms you will be looking for when verifying encoder operation.

Operating instructions

MCC configuration for motorBench projects

Recommended compiler and MCC peripherals can be found in the motorBench 2.55.0 Release Notes. Depending on which dsPIC device is being used, select the appropriate MCC environment as shown in the table below.

Device family being used	Use
dsPIC33EP	MCC Classic

Device family being used	Use
dsPIC33C	MCC Melody
dsPIC33A	MCC Melody

MCC Classic

MCC Classic is required when using motorBench® Development Suite with the supported dsPIC33EP device. In this case, starting with a sample project is strongly recommended. The sample motorBench project sets up all peripheral, system, compiler, and linker settings. Navigate to the [release repository](#)¹² and choose one of the projects below based on the development board that you want to use.

Device	MCLV-2
dsPIC33EP256MC506	sample-mb-33ep256mc506-mclv2.X

MCC Melody

motorBench® Development Suite automatically configures MCC Melody to set up the required peripherals while using one of the supported dsPIC33C or dsPIC33A devices. A sample project is not necessary in this case.

The Motor Control Application Framework requires certain compiler and linker settings; without these settings it may not run correctly. Refer to the motorBench Release Notes for compiler and linker settings.

Using MCC Classic

For dsPIC33EP devices, it is strongly recommended to start with a sample MPLAB X project – the compiler, linker, and MCC System and Peripheral settings have been preset for use with the Motor Control Application Framework.

1. Unzip one of the motorBench sample projects onto your computer.
2. Open the sample motorBench project in MPLAB X.
3. Right-click the project and click **Set as the main project** in MPLAB X.
4. Right-click and open project properties. Select an in-circuit programmer (see the motorBench v2.55.0 Release Notes for recommended in-circuit programmers) and XC16 / XC-DSC compiler version.
5. MCC can be launched from the MPLAB® X Tools menu under the Embedded selection, or by clicking on the MCC icon in MPLAB X Toolbar.
6. MCC help can be invoked by clicking on the 

Using MCC Melody

1. Create a New Project.
 - Go to **File** → **New Project...**
 - Select **Application Project(s)**, click **Next**.

¹² <https://github.com/microchip-pic-avr-solutions/motorbench-release-collateral>

- Choose the desired **Device** from one of the dsPIC33C or dsPIC33A families.
- 2. Select programming tool and compiler.
 - Select a **Tool** to use for programming the device from the drop-down menu.
 - If the programming tool is not connected, it will not be available from the drop-down.
 - Connect programming tool to proceed.
 - Select an **XC16 / XC-DSC compiler** version.
- 3. Set **Project Name** and **Project Location**.
 - Select **Set as main project**.
 - Unselect **Open MCC on Finish**.
- 4. Verify project properties.
 - Right-click the project → **Properties**.
 - Confirm **Connected Hardware Tool** and **Compiler Toolchain**.
- 5. Open MPLAB Code Configurator (MCC) Content Manager (CM).
 - Click the **CM** icon in the toolbar.
- 6. Select the compatible content versions in MCC CM.
 - Click **Load Manifest**, select the correct `.yaml` file from the motorBench release repository for your device.
 - Verify the content version(s), click **Apply**.
 - Wait for the Content Manager to finish updating the content.
- 7. Close CM.
- 8. Open MCC by clicking the **MCC** icon in the toolbar.
- 9. Add **motorBench® Development Suite** to project resources to get started.

The Motor Control Application Framework requires certain compiler and linker settings; without these settings it may not run correctly. Refer to the motorBench Release Notes for compiler and linker settings.

Configuration

1. Select motorBench® Development Suite in Project Resources under Libraries.
 - The sample projects, if applicable, already have the motorBench library added.
 - If the motorBench library is not present in Project Resources, then add it by locating it in Device Resources under Libraries, and double-clicking on it.
2. If you haven't registered, you will be given the option to [register motorBench¹³](#), or enter a 30 day trial period. For registration steps on an offline machine please contact your local Microchip representative.
3. motorBench® Development Suite has three separate views
 - a. Configure: Allows user to configure the motor and board. Users have the option to measure the electrical and mechanical parameters of their motor. Users can export motors and boards from their project.
 - b. Tune: If desired, tuning parameters for the motor current and velocity loops can be adjusted. This is an optional step.
 - c. Customize: Allows users to customize MCAF parameters that are used in code generation.
4. Once you register, the gray margin will show "Registered", otherwise it would show "Not yet Registered"
5. You can navigate between the "Configure", "Tune" and "Customize" stages using their respective buttons.
6. Once all the components are fully defined and there are no configuration errors, a completed puzzle icon will appear with the status "Ready to Generate". If the puzzle icon doesn't appear complete, check if there are any messages present in the "Notifications[MCC]" tab below that prevent completion, and resolve them.

¹³ <https://www.microchip.com/en-us/solutions/motor-control-and-drive/motorbench-development-suite#:~:text=Registration%20Instructions>

Board configuration

While motorBench has integrated support for Microchip development boards, it also supports custom boards that users can define using text files serialized in YAML format.

When starting a new project, use one of these options:

1. Select from one of the supported Microchip development boards.
2. Import one of the board definition files shared on the [motorBench release repository](#)¹⁴.
3. Import one of the existing board definition files specifically created for a given custom board.

Selected board configuration is serialized and saved within the MCC project file.

Default configuration

Upon creating a new project, motorBench® Development Suite v2.55.0 defaults to the development board and a supported PIM/DIM depending on the selected device in the project, as shown in the table below.

Device family being used	Default hardware configuration
dsPIC33EP	dsPICDEM™ MCLV-2 Development Board & dsPIC33EP256MC506 Motor Control External Op Amp PIM
dsPIC33C	MCS MCLV-48V-300W Development Board & dsPIC33CK256MP508 Motor Control DIM
dsPIC33A	MCS MCLV-48V-300W Development Board & dsPIC33AK128MC106 Motor Control DIM

Supported development boards

motorBench® Development Suite has integrated support for the following Microchip development boards:

- dsPIC33CK Low Voltage Motor Control Board ([DM330031](#))¹⁵

Boards supporting Plug-in Modules (PIMs):

- dsPICDEM™ MCLV-2 Development Board ([DM330021-2](#))¹⁶

Boards supporting Dual Inline Modules (DIMs):

- MCS MCLV-48V-300W Development Board ([EV18H47A](#))¹⁷
- MCHV-230VAC-1.5kW Motor Control High-Voltage Development Board ([EV78U65A](#))¹⁸

¹⁴ <https://github.com/microchip-pic-avr-solutions/motorbench-release-collateral>

¹⁵ <https://www.microchip.com/en-us/development-tool/DM330031>

¹⁶ <https://www.microchip.com/en-us/development-tool/DM330021-2>

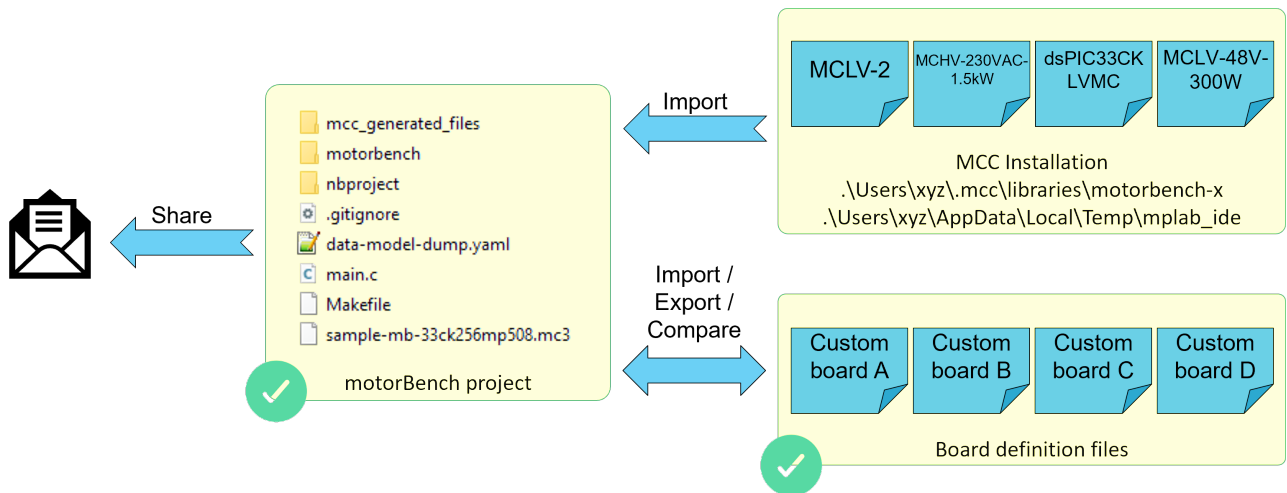
¹⁷ <https://www.microchip.com/en-us/development-tool/EV18H47A>

¹⁸ <https://www.microchip.com/en-us/development-tool/ev78u65a>

Support for custom boards

motorBench® Development Suite supports custom motor drive hardware using one of the following workflow steps:

1. Export board data from a project into a board definition file
2. Import board data from a board definition file into the project
3. Compare the selected board in a project with one of the supported Microchip development boards



See [MCAF user's guide](#)¹⁹ chapter "Custom board support" for more information.

Creating and using a custom board definition file

1. Obtain a copy of the starter board file from the [motorBench release repository](#)²⁰.
2. Open it in a text editor like VS Code
3. Make necessary changes to the starter board definition while maintaining the structure, including adding new sections to completely define the custom board.
4. Save the custom board definition file and close the text editor.

On the Configure page of motorBench, use the 'Import Board' button to import the custom board definition file to begin using it.

i motorBench allows a maximum of one custom board to be saved into a project. Importing another custom board will overwrite the existing custom board definition in the project.

Making changes to custom board definition in an existing project

1. Open the project with MPLAB X IDE, launch MCC and open motorBench
2. On the Configure page, select the custom board from the drop-down menu.
3. Select 'Export board' and save the board definition data into a file on the local disk.
4. Open this board definition file in a text editor and make necessary changes.
5. Select 'Import board' and import the modified board definition file.

¹⁹ <https://microchiptech.github.io/mcaf-doc/latest/>

²⁰ <https://github.com/microchip-pic-avr-solutions/motorbench-release-collateral>

 Board parameters displayed on the Configure page of motorBench are presented in a read-only format in this release of motorBench.

Comparing board definition

motorBench® Development Suite includes a feature to compare the currently selected board definition with one of the supported Microchip development boards to help identify discrepancies or troubleshoot issues with board formatting.

1. Click the **Compare Boards** button to open the **File Difference Viewer**.
2. Select one of the supported Microchip boards and a compatible PIM/DIM to compare the current board against.
3. Click the 'Compare' button to launch a diff window that highlights the differences in between the two boards under comparison.

System parameters

System parameters (for example, **PWM frequency** and **dead time**) are application properties that depend on the **board definition file** used in the motorBench project. View and edit system parameters on the **Configure** page in motorBench® Development Suite.

When importing a new board definition or selecting one from the drop-down menu, record any manual changes made on the **Configure** page and re-verify them after importing a new board file, as all system parameters will be reset to the selected board definition's default values.

After any system parameter from the current board-definition default has been manually changed, follow these steps when loading a new board definition:

1. When prompted with “**One or more system parameters will be updated to support the selected board.**”, select **OK** to update system parameters to the default values for the selected board.
2. Select **Cancel** to keep the current system parameters and board selection unchanged.

Set DC Link Voltage for MCHV-230VAC-1.5kW Board

Be sure to set the **Nominal DC link voltage** on the **Configure** page for the MCHV-230VAC-1.5kW board. Calculate using the function or refer to the table below for common values:

$$V_{DC\ link} = V_{RMS} \times \sqrt{2}$$

Nominal AC input voltage (V_{RMS})	Value to be entered in Configure → System Parameters → Nominal DC link voltage (V)
110	155.5
120	169.7
220	311.1

Nominal AC input voltage (V_{RMS})	Value to be entered in Configure → System Parameters → Nominal DC link voltage (V)
240	339.4

Motor import and export

motorBench® Development Suite includes motor data for the following five reference motors:

Reference motor	For use with
hurst300	Low voltage boards example: MCLV-2, dsPIC33CK LVMC, MCLV-48V-300W
hurst075	Low voltage boards example: MCLV-2, dsPIC33CK LVMC, MCLV-48V-300W
elra-ipm80s-24v	Low voltage boards example: MCLV-2, dsPIC33CK LVMC, MCLV-48V-300W
linix-45zwn24-40	Low voltage boards example: MCLV-2, dsPIC33CK LVMC, MCLV-48V-300W
leadshine400	High voltage boards example: MCHV-230VAC-1.5kW

Importing a motor

When a new project is created for the first time, there is no motor selected. Click on the 'Import Motor' button to select one of the motor data files included within motorBench® Development Suite, or a motor data file for a custom motor.

Select the desired motor, and click 'Open'. The motor will be imported into motorBench® Development Suite. motorBench® Development Suite is now ready to generate code.

Exporting a motor

Once an existing motor is imported, users may choose to change its parameter values.

Motor and your project

Note that once a motor is imported into motorBench® Development Suite, there is no link between the project and the file from which motor parameters were imported. 'Output - MPLAB® Code Configurator' window will show a log event with details of the filesystem path from which motor parameters were imported, but any changes made in the UI are local to the project and do not affect the original file.

All the motor parameters displayed under the 'Active Values' column are editable and changes to these parameters will be saved to the main project. Parameters listed under the 'Measured Values' column are automatically calculated by the motor parameter measurement process.

All numeric values are shown in scientific notation. When the mouse pointer is hovered over each of the individual parameters, the text box will display the value in double-precision floating point. Currently, motorBench supports entering motor parameter values in the units shown next to each parameter.

Users can choose to export the motor data if they need to use the same motor for other/new projects. Clicking the 'Export Motor' button will bring up a dialog box that will allow users to save a copy of the motor data, including any edits made to motor parameters in the motorBench UI, to a file.

 Only the 'Active Values' in the 'Electrical and Mechanical Parameters' section are exported to the file. 'Measured Values' are saved only in the main project.

Working with custom motors

To start working with a custom motor, import the configuration for one of the reference motors. Then, update all of the identification and nameplate data from the datasheet or specification sheet of the custom motor. After this, update the electrical and mechanical parameters of the motor under 'Active Values' column by selecting



button after motor parameter measurement (Self-Commissioning) completes, or by looking up these values from the motor datasheet.

Save all changes to the project by pressing the  button in the MPLAB X IDE toolbar. After this, the user may choose to export the configuration of this new motor as described in the section above to a file for future reference.

Editing exported files

Manual editing of the motor data file is not recommended. If needed, import the motor data file into the project, make the necessary edits and then export the motor data to the same/another file.

Motor parameter measurement

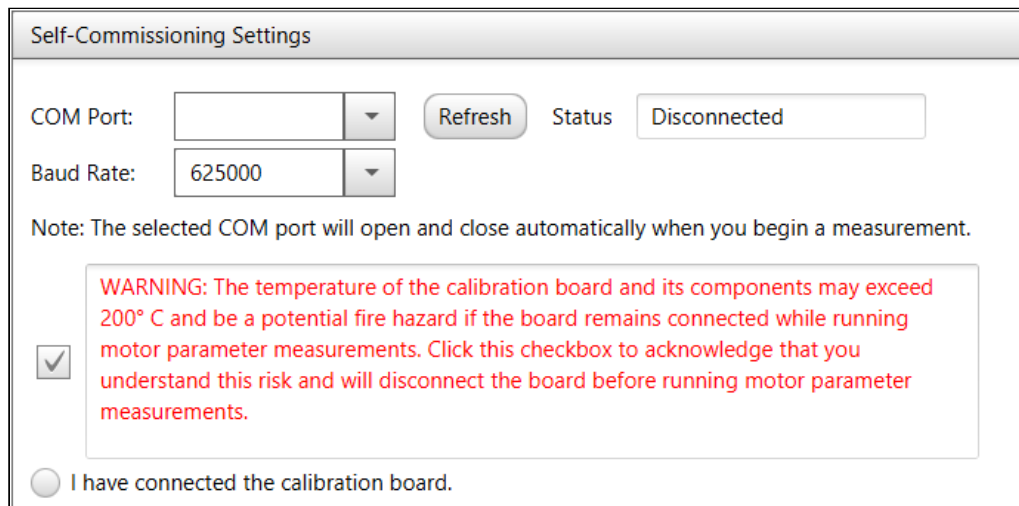
Also referred to as Self-Commissioning, this feature of motorBench can automatically measure electrical and mechanical parameters of a wide range of custom motors.

Currently, this feature is supported only on the dsPIC33EP256MC506 External Op Amp PIM with MCLV-2 development board.

Operating instructions

After importing a motor, click on the "Measure Now" button in the motor's Electrical and Mechanical Parameters group to open up the motor parameter measurement window.

1. Before beginning the measurement, make sure that a programming tool and a compiler are selected in the project properties.
2. Select the appropriate COM port from the combo box. Click 'Refresh' to update the list of COM ports.
3. Acknowledge the heat hazard warning. This is necessary to proceed with the measurement.



The dialog box titled "Self-Commissioning Settings" contains the following elements:

- COM Port:** A text box with a dropdown arrow, followed by a "Refresh" button and a "Status" label showing "Disconnected".
- Baud Rate:** A text box with the value "625000" and a dropdown arrow.
- Note:** "The selected COM port will open and close automatically when you begin a measurement."
- Warning Box:** A red-bordered box containing the text: "WARNING: The temperature of the calibration board and its components may exceed 200° C and be a potential fire hazard if the board remains connected while running motor parameter measurements. Click this checkbox to acknowledge that you understand this risk and will disconnect the board before running motor parameter measurements." Below this text is a checked checkbox.
- Radio Button:** An unchecked radio button labeled "I have connected the calibration board."

i The default communication baud rate setting during motor parameter measurement is 625000. However, the RS232 transceiver devices used in MCLV-2 development board (i.e. MAX3232CUE) are specified to operate reliably at baud rates up to 120kbps. Due to this limitation, while using MCLV-2 development board during motor parameter measurement, in case of serial communication related issues, retry measurement after reducing the communication baud rate to a lower setting.

4. To calibrate, connect a [valid calibration load](#) (see page 37) (see Appendix for recommendations) and click the "I have connected the calibration board" radio button. Click 'Calibrate'.
5. To run the motor parameter measurements,
 - a. Disconnect the calibration load and connect the motor instead.
 - b. Click the "I have disconnected the calibration board" and "I have connected the motor" radio buttons.
 - c. Click the 'Start' button to begin measurement. Measured values will sequentially appear next to each parameter as and when measurement complete.
6. The measured values are updated in the 'Measured Values' of the Motor's Electrical and Mechanical

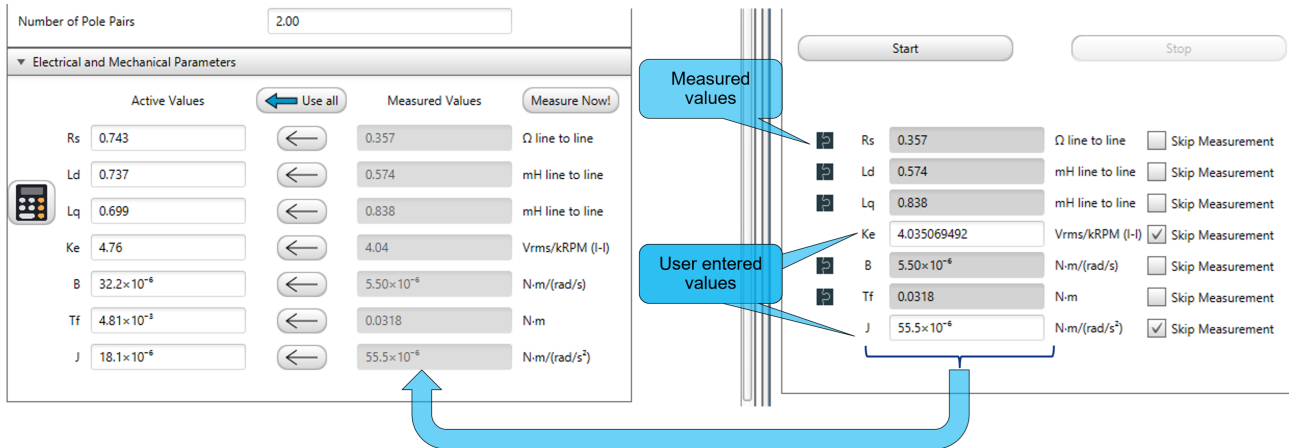
Parameters section. Click on the  icon to set a measured value as the active value. The  Use all button can be used to set all measured values as active values. The 'Active Values' are used for Autotuning and code generation.

Selective measurements feature

Motor parameter measurement feature of motorBench allows users to optionally skip one or more motor parameter measurements in the measurement sequence using the 'Skip Measurement' option. This feature can be useful in cases where the motor parameter measurement feature fails to measure specific motor parameters, or if the measurements are not accurate enough.

To use this feature, select the 'Skip Measurement' checkbox and then manually enter a parameter value into the text box for measurements that you want motorBench to *not* measure before clicking on the 'Start' button to start the parameter measurement process. motorBench will use the user-entered parameter values for internal calculations while measuring the remaining motor parameter values. Once the measurement is complete,

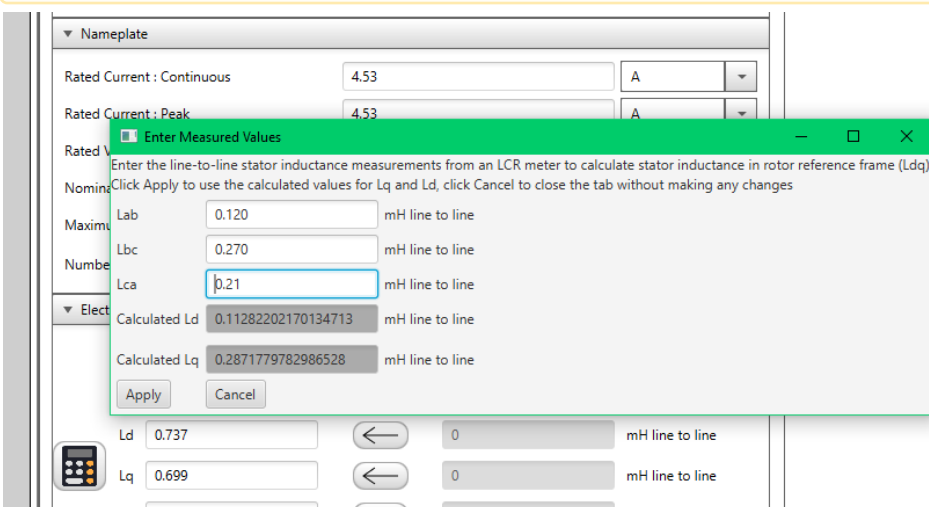
motorBench will reflect the measured as well as user-entered parameter value under the 'Measured values' column of the Configure page.



Inductance calculator widget

This widget helps to calculate the rotor reference frame (L_{dq}) inductance values from line to line measurements of three phase stator reference frame (L_{abc}) inductance values, typically obtained using an LCR meter. Clicking on the  icon in the Configure page opens up a widget window. The widget will instantly calculate L_d and L_q as soon as values of L_{ab} , L_{bc} and L_{ca} are entered. Click on 'Apply' to insert the calculated values of L_d and L_q into the 'Active Values' column of motor parameters.

⚠ Inductance measurements should generally be performed with excitation signals in the 1 kHz - 20 kHz range. Before using an LCR meter, check to see what frequency it uses for measurement. Use of low-frequency (≤ 100 Hz) measurement may give incorrect readings because of motor shaft vibration and generated back-emf. Use of high-frequency measurement (≥ 100 kHz) may give incorrect readings because of parasitic capacitances.



Common types of motor parameter measurement errors and reasons they can occur

General guidance: The usual cause of errors during motor parameter measurement is due to incorrect definition of motor and/or board specifications in the "Configure" page, specifically these parameters:

1. Configure → PMSM Motor → Rated current: Continuous
2. Configure → PMSM Motor → Number of pole pairs
3. Configure → PMSM Motor → Nominal speed
4. Configure → Board → Voltage Source → Output

Table below provides some more specific examples of typical issues and their possible causes.

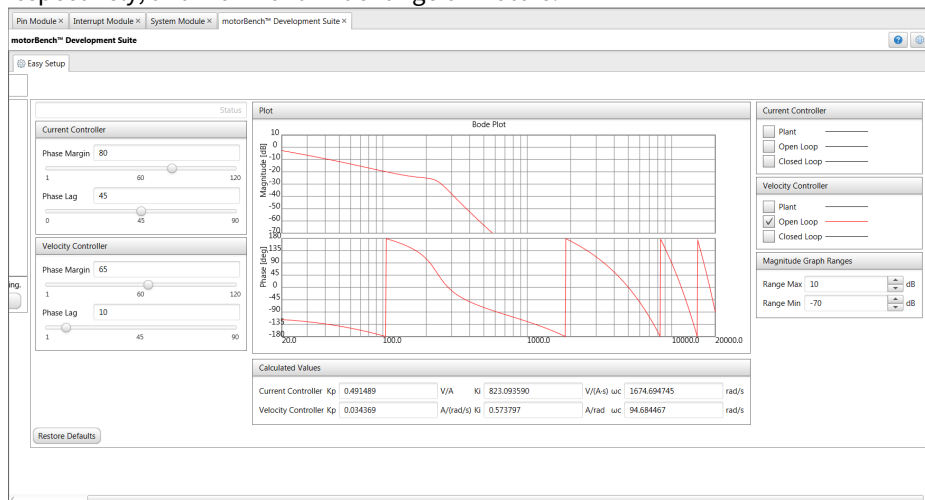
Symptom	Possible causes	Possible fix
Hardware over-current fault during R, Ld or Lq measurements (Fault Code #10)	Rated current specification of the motor is close to / higher than the full scale current range of the board. In this case, with motors that have relatively low values of electrical time constant, current ripple can trigger the over-current comparator on the board.	Retry the measurement after reducing the value in Configure → PMSM Motor → Rated current: Continuous to a lower value.
Motor stalls during inertia measurement with a Fault Code #107	The specified nominal speed of the motor is either too low or too high for the given voltage source output at the inverter.	Correct the value in Configure → PMSM Motor → Nominal speed and retry the measurement.
Motor stalls during Ke measurement with a Fault Code #10, #102, #103, #104, #105 or #106	If the test motor has a relatively low value of stator inductance (typical Ld and Lq ~ 100µH or less), then transients in the current ripple can trigger the hardware over-current fault comparator causing the motor to stall during startup.	Increase the Measure → Motor Parameters → Current Controller → Phase Margin (slider) from its default value of 95° in 5° increments up to a maximum value of 115° and retry the measurement.

Symptom	Possible causes	Possible fix
	If the test motor has a relatively high value of stator inductance (typical L_d and $L_q \sim 1\text{mH}$ or more), then the current controller may need to be more aggressive during the startup phase in order to be able to suppress any open loop oscillations that may cause disturbances in the current control loop.	Decrease the Measure → Motor Parameters → Current Controller → Phase Margin (slider) from its default value of 95° to 91° and retry the measurement.
	If the test motor has significant inertia (typical $J \sim 100\mu\text{Nm}/(\text{rad}/\text{s}^2)$ or more) then, Measure → Motor Parameters → Approx. Spin Down Time parameter may be set to value that is too low for the given motor.	Increase the value that is set in Measure → Motor Parameters → Approx. Spin Down Time and retry the measurement. This will allow the measurement algorithm to wait for the motor to spin down completely before attempting to restart the motor in between measurements.
Measurement fails during R measurement with a message "Unable to detect a motor on the board inverter output. Please verify that the leads of the motor under test are connected securely to the board inverter output terminals."	All three phase wires of the test motor are not correctly connected to the inverter output terminals.	Verify that all three phase wires of the motor are connected correctly to the inverter output terminals.
	Test motor has a stator inductance value that is out of supported measurement range.	-
	Voltage source that is connected to the inverter stage is not powered on.	Verify if the inverter stage is connected to a voltage source that is powered on.

Autotuning

1. Click the 'Tune' button to go to the Tuning stage. There is no required user action in this stage. You can check out the Bode plots or use different settings for phase margin or PI phase lag at crossover if you wish to do so.

2. The defaults for Phase Margin/Phase Lag are 80/45 and 65/10 for current and velocity controllers, respectively, and work for a wide range of motors.



Managing errors in autotuning

Autotuning may fail in certain extreme cases:

- Motor parameters are extreme values, either because the motor parameter measurement process did not work correctly or because the motor itself is unusual
- Tuning parameters (phase margin and PI phase lag at crossover) are extreme values
- Customize page parameters are extreme values

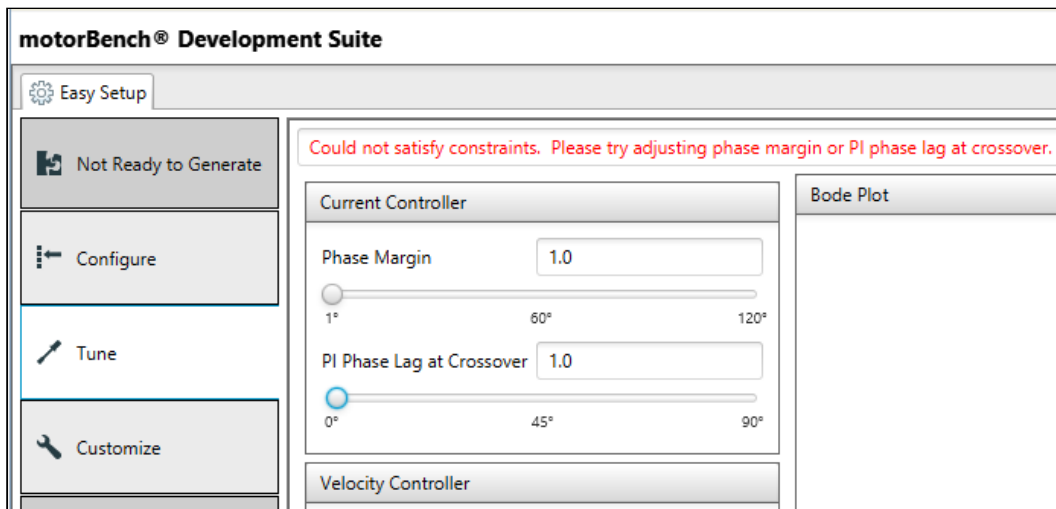
Important things to know are

- How to recognize that an error has occurred
- Common types of autotuning errors and reasons they can occur
- How to report specific failure details to Microchip for assistance

How to identify autotuning errors

If an error has occurred during autotuning, the following will occur:

- motorBench will show "Not Ready to Generate"
- The Tune page will display a short status message in red, and no Bode plot will be drawn on the graphs
- The Notifications [MCC] tab will show a list of problems (in most cases)



Notifications	Output - MPLAB® Code Configurator	Notifications [MCC] x	Pin Manager: Grid View
Category	Source	Type: ALL	Description
	DMA	HINT	Configure EXI_INT module for DMA Channel: 3
	motorBench® Developm...	WARNING	motorBench® Development Suite configuration is not yet complete. We do not recommend generating code at this time, since it is likely to produce erroneous results.
	motorBench® Developm...	WARNING	Unable to tune the velocity or current loop: Could not satisfy constraints. Please try adjusting phase margin or PI phase lag at crossover. Refer to the motorBench Release Notes or MCAF User's Guide, available from the Microchip website, for further guidance on this issue.
	ADC1	HINT	To use OA2 output for CH123SA, select CMP2 and set it to GRAMP mode.

Common types of autotuning errors and reasons they can occur

Symptom	Possible cause	Possible fix
Tune page: "No motor imported"	A motor has not been imported on the Configure page.	Import a motor on the Configure page.
Tune page: "Could not satisfy constraints. Please try adjusting phase margin or PI phase lag at crossover."	Tune page parameters (phase margin or PI phase lag) may be unusually low or high	Change tuning parameters to be closer to their default values. (The "Restore Defaults" button will set these values back to their defaults.)
	Estimator parameters on the Customize page may be unusually low or high	Change estimator parameters to be closer to their default values. (Customizable estimator parameters are AN1292 PLL time constant and bandwidth and Quadrature encoder tracking loop time constant — all of which are shown only if the "Show advanced parameters" checkbox is checked.)

Symptom	Possible cause	Possible fix
	Certain other parameters (minimum operating velocity) on the Customize page may be unusually low or high	Change parameters to be closer to their default values.
Tune page: "Could not evaluate transfer function."	May occur if minimum operating velocity is zero.	Choose a minimum operating velocity greater than zero.
Other symptoms (for example, "Could not locate transfer function logic" or "Could not construct transfer function")		Please report to Microchip – these are unexpected errors.

How to report specific failure details to Microchip for assistance

If contacting Microchip staff for assistance, copy the entire text of the Tune page status message and the description of any MCC warnings – not just a screenshot. (Click on the table cell in the MCC Notifications pane, so that it is highlighted as shown below, and press Ctrl+C to copy the text to the system clipboard.)

Type:	Description
WARNING	not recommend generating code at this time, since it is likely to produce erroneous results. Unable to tune the velocity or current loop: Could not evaluate transfer function. Refer to the motorBench Release Notes or MCAF User's Guide, available from the Microchip website, for further guidance on this issue. Failed to findRoot.
HINT	To use OA2 output for CH123SA, select CMP2 and set it to OPAMP mode

This provides important clues that may indicate the cause of the problem and how to address it.

Customize

The Customize page allows users to modify parameters used for MCAF code generation. This is an optional step to improve the behavior of the generated code for some motors; none of the parameters on the Customize page need to be modified in order to generate code. The default values found in the Customize page are good for most motors.

Position and velocity estimator selection

One estimator must be selected as the "primary" estimator used for commutation and velocity feedback. Additional estimators may be selected as secondary estimators for comparison of angle and velocity information.

Refer to the [MCAF User's Guide](#)²¹ for more detailed information about the different position and velocity estimators.

Advanced parameters

There are several dozen customizable parameters, but most of these are considered "advanced" and are used only in rare cases to solve specific problems. Only a handful of parameters are shown by default; to show all parameters, click on the checkbox marked "Show advanced parameters" at the top of the Customize page.

Normalized parameters

Many parameters are normalized to motor or system values, in order to ensure consistent default behavior. For example, the startup current I_{q0} is specified as a fraction of maximum current. The Customize page will display the actual value in real-world engineering units as a visual and computational aid.

▼ Motor startup

Note: See sample graph below, which illustrates many of these startup parameters.

Current I_{q0} $\times I_{max}$ = 1.7175 A

Nominal startup current, normalized to maximum current I_{max} , where I_{max} = minimum of motor and drive continuous ratings

Min accel time t_{acc} $\times L/R$ = 241.51 ms

Determines the minimum allowable acceleration time, which affects the maximum acceleration during startup. Acceleration rates are determined using motor mechanical parameters, and can be slower, but not faster than this.

Each of these parameters has a default value. A "restore" icon  is displayed next to any parameter that has been modified. To view the default value, hover over the restore icon. To restore the default value, click on the restore icon.

Advice

The "Advice" section in the Customize page provides guidance on the effect of certain system quantities such as ripple current, as shown below:

Ripple current at maximum DC link voltage

$$I_R = \frac{V_{DC} T_{PWM}}{12L} = 0.1318 \times I_{max}$$

$I_R < 0.2I_{max}$	Low ripple current (< 1.3% additional I^2R loss)
$0.2I_{max} \leq I_R < 0.4I_{max}$	Moderate ripple current (< 5.3% additional I^2R loss)
$I_R \geq 0.4I_{max}$	High ripple current ($\geq 5.3\%$ additional I^2R loss)

I_R describes the worst-case peak amplitude of ripple current, which occurs when the three motor phases are switching at some permutation of (0%, 50%, 100%). Ripple current can approach this value at high modulation indices. The RMS value of ripple

This information is provided to highlight specific quantities that may cause problems in some systems. These are general guidelines and may not apply to all motor control systems.

²¹ <https://microchiptech.github.io/mcaf-doc/latest/>

Additional information

Refer to the [MCAF User's Guide](#)²² for more detailed guidance on parameter customization.

Generate code

1. To generate code, click "Generate" under the Project Resources section. Look for the generation progress under the MPLAB® Code Configurator console window.

Once generation is complete, you will see a banner comment as shown here:

Notifications	Output	Notifications [MCC]	Pin Manager: Grid View
Project Loading Warning MPLAB® Code Configurator			
16:34:59.647	INFO:	*****	
16:34:59.648	INFO:	Generation complete (total time: 27250 milliseconds)	
16:34:59.648	INFO:	*****	
16:34:59.648	INFO:	Generation complete.	

You will also be able to see the generated files added to your project.

2. If you want to run generation again, and have not made any manual edits, consider removing previously-generated files under the mcc_generated_files/motorBench directory. This will avoid having to merge newly-generated files with earlier-generated files.
3. You are now ready to build and run the project, and spin the motor.

Managing errors in code generation

Code generation may fail in certain extreme cases:

- Motor parameters are extreme values, either because the motor parameter measurement process did not work correctly, or because the motor itself is unusual
- Tuning parameters (phase margin and PI phase lag at crossover) are extreme values
- Customize page parameters are extreme values

Important things to know are

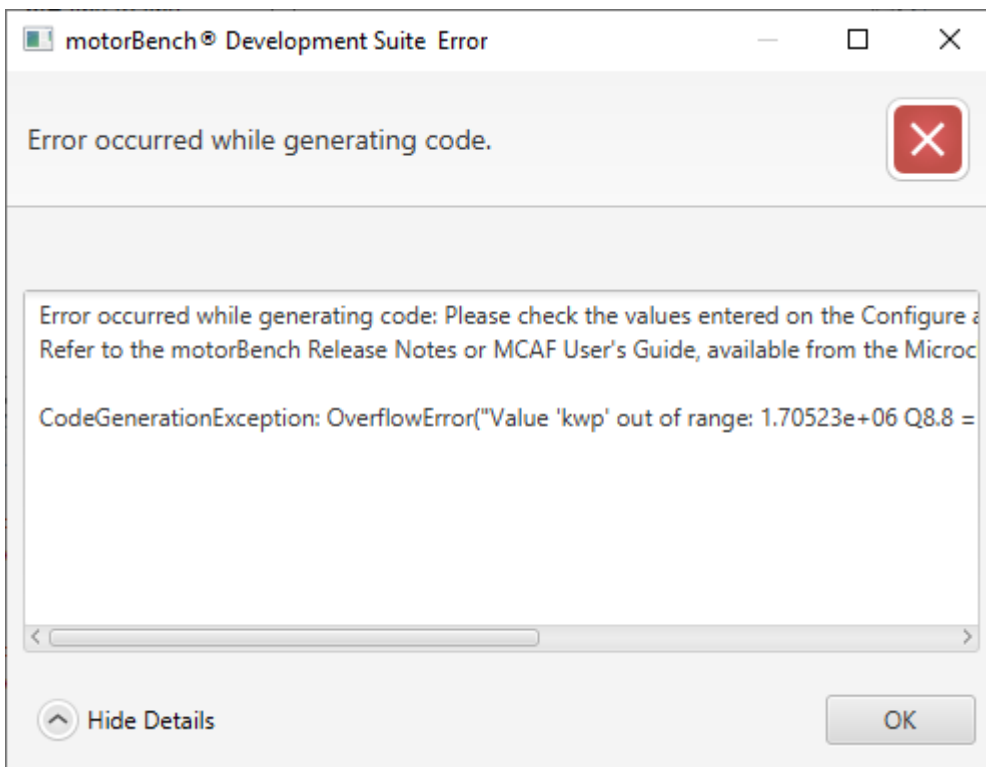
- How to recognize that an error has occurred
- How to report specific failure details to Microchip for assistance
- Common types of code generation errors and reasons they can occur

How to identify code generation errors

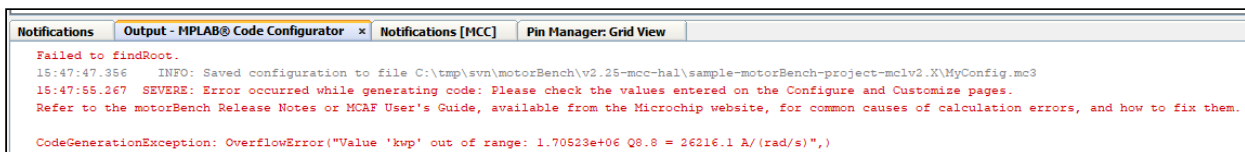
If an error has occurred during code generation, the following will occur:

- A dialog box will pop up with an error message:

²² <https://microchiptech.github.io/mcaf-doc/latest/>



- MPLAB® Code Configurator console window will have a SEVERE category message like shown below:



Common types of code generation errors and reasons they can occur

General guidance: The usual cause of code generation errors is a motor parameter that is extremely high or extremely low. See the section "Motor Control Limitations" which discusses ranges of motor parameters that work well with motorBench® Development Suite. There are a few reasons a motor parameter could cause a code generation error:

- The parameter has not been measured or entered correctly. (For example, a datasheet states 50μH but is entered as 50mH = 0.05H.)
- The motor has unusual motor parameters. (Slotless motors may fall into this category, as they have very low inductance.)
- The motor is not well-matched to the motor drive circuitry, for example use of a 6V motor with a 24V motor drive.

Similarly, one of the Customize page parameters may also be at an extremely high or low value.

Looking at the error message may help to diagnose and check for a fix.

Symptom: category	Symptom: specific message	Possible cause	Possible fix
ZeroDivisionError		One of the motor parameters is zero. (This should never be the case.)	Make sure none of the motor parameters are zero.
ZeroDivisionError		One of the Customize page parameters is zero	Check whether there are Customize page parameters that are zero, but should be greater than zero.
CodeGenerationException: OverflowError	kwp out of range	This is the velocity loop proportional gain. Out-of-range errors can occur if the inertia (J) is very high.	Make sure the inertia value is reasonable. An increase or decrease in velocity loop phase margin in the Autotuning page may be required for high-inertia motors. (Increasing phase margin generally lowers kwp whereas decreasing phase margin raises it.)
CodeGenerationException: OverflowError	velocitySlewrateLimitDecel out of range	May occur if Coulomb friction torque (Tfr) is out of range.	Make sure the Coulomb friction value is reasonable.
CodeGenerationException: OverflowError	normLsdt out of range	May occur if inductance (Ld and/or Lq) is out of range.	Make sure the motor inductance values are reasonable and well-matched to the motor drive.
CodeGenerationException: OverflowError	normRs out of range	May occur if resistance (Rs) is out of range.	Make sure the motor resistance values are reasonable and well-matched to the motor drive.

Symptom: category	Symptom: specific message	Possible cause	Possible fix
CodeGenerationException: OverflowError	keInverse out of range	Specified nominal speed of the motor is too low for the given motor design and full scale voltage range of the selected board.	1. Increase the nominal speed specification of the motor in the Configure page within reasonable limits specified in its datasheet 2. Reduce the full scale voltage range of the board by modifying its DC bus voltage sense circuit.
CodeGenerationException: OverflowError	dILimitLs out of range	Very high amplitude of estimated current ripple for the given motor and the DC bus voltage.	1. Increase PWM frequency 2. Switch to another estimator E.g., ATPLL
CodeGenerationException: OverflowError	(Other messages)	A parameter on the Customize page is at an extreme value	Recheck values on the Customize page.
Other errors			Please report to Microchip – these are unexpected errors.

How to report specific failure details to Microchip for assistance

If contacting Microchip staff for assistance, copy the entire text of the dialog box or MCC Output window — not just a screenshot.

This provides important clues that may indicate the cause of the problem and how to address it.

Building code

At this point, work in motorBench® Development Suite is complete. In MPLAB X, click the Clean button and then the Run button to build and program the device.



Reminder

The Motor Control Application Framework requires certain compiler and linker settings; without these settings it may not run correctly. Refer to the motorBench Release Notes for compiler and linker settings.

Running the Motor Control Application Framework

Directions for using MCAF with each of the supported boards are slightly different.

MCLV-2

- Press button S2 to start/stop the motor.
- Press button S3 to reverse the direction of rotation.
- Turn the potentiometer to control speed.

dsPIC33CK LVMC and MCLV-48V-300W

- Press button SW1 to start/stop the motor.
- Press button SW2 to reverse the direction of rotation.
- Turn the potentiometer to control speed.

MCHV-230VAC-1.5kW

- Press BUTTON 1 (SW2) to start/stop the motor.
- Press BUTTON 2 (SW3) to reverse the direction of rotation.
- Turn the potentiometer to control speed.

In the event of an error, both LEDs will flash together to indicate an error code; see the 'Error Codes' documentation in the [MCAF User's Guide](#)²³ for further information. The component number for the LEDs used for error indication on each board are listed below.

Board	LED 1	LED 2
MCLV-2	D2	D17
dsPIC33CK LVMC	LD10	LD11
MCLV-48V-300W	LD2	LD3
MCHV-230VAC-1.5kW	D15	D16

Real-Time Diagnostics

The Motor Control Application Framework includes out-of-the-box support for [X2Cscope](#)²⁴, a third-party plugin for MPLAB X which facilitates real-time diagnostics. X2Cscope is available in the same **Available Plugins** tab used to install motorBench® Development Suite.

²³ <https://microchiptech.github.io/mcaf-doc/latest/>

²⁴ <https://x2cscope.github.io/>

Troubleshooting issues with some motors

We have observed some issues during our testing under the following circumstances:

1. Starting up small motors with large inertia loads
2. Starting up certain motors with low speed reference
3. Instability in current loop

These are some solutions that may work for some motors:

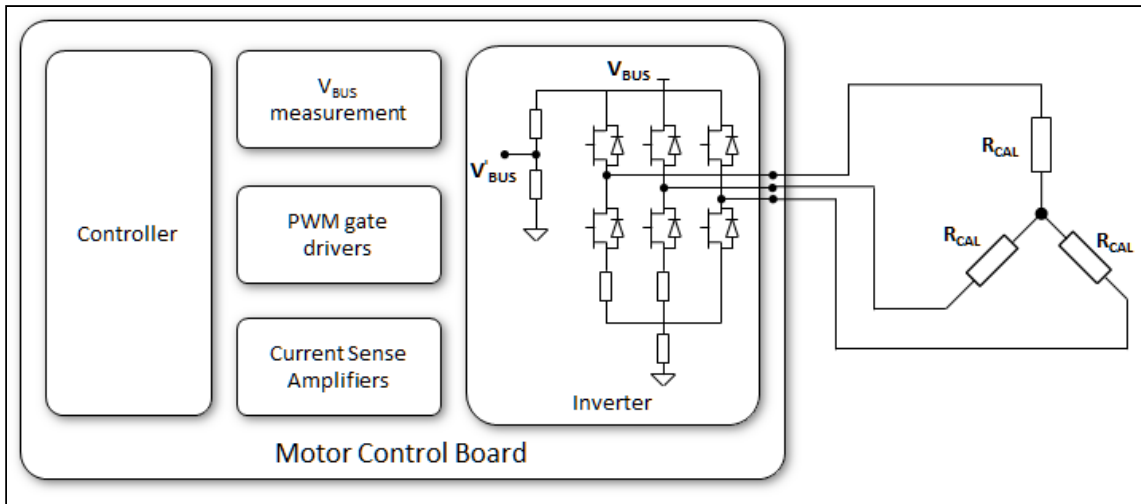
1. Starting up small motors with large inertia loads using the classic startup method may have a difficult time with the transition from forced commutation to closed-loop commutation. Use the Weathervane startup method in these circumstances.
2. Starting up certain motors with low speed reference fails to start. This may be due to either an issue in the startup method or the behavior of the sensorless estimator during the transition to closed-loop commutation. Workarounds for this issue include using the Weathervane startup method, ensuring that the speed command is not at the low end of its range, or reducing the DC link voltage if possible.
3. If instability in current loop is observed an increase of 5° or 10° in current loop phase margin may help.

Appendix

Motor parameter measurement - Calibration load specifications

The board calibration process analyses the hardware to measure any deviation from its design values of board parameters. This will help improve the accuracy of motor parameter measurements. The board calibration step of motor parameter measurement is optional. However, it is recommended that at the least, it is run once every time there is a change in the development board.

The board calibration process requires three equal-value resistors to be connected to the three-phase inverter output as shown in the diagram below.



The following equation provides the recommended range of calibration resistance values based on parameters of the board that is being calibrated:

$$\frac{V_{BUS}}{0.20 \times I_{fullscale} \times 2} \leq R_{CAL} \leq \frac{V_{BUS}}{0.80 \times I_{fullscale} \times 2}$$

where $I_{fullscale}$ and V_{BUS} are defined as follows:

- $I_{fullscale}$ — full-scale current for the board
- V_{BUS} — voltage applied to the inverter's three-phase bridge.

During the calibration process, the specialized firmware applies a series of voltage pulses on the calibration resistors and measures the resulting current. The pulse train has the following properties:

- Amplitude of V_{BUS}
- Pulse train duration of 2 ms with each pulse lasting 50 μ s
- Duty cycle of 46%

Since the pulses applied have an amplitude of V_{BUS} , the calibration resistors must be voltage rated to handle at least half of V_{BUS} . The pulse train also imposes resistor size requirements due to the energy in the pulses that need to be dissipated with a nominal temperature rise to prevent resistance changes.

Factoring all the above requirements, for the dsPICDEM™ MCLV-2 Development Board, use a 10 Ω , 21 millijoules resistor.

In order to ensure accurate calibration, use resistors with

- 1% tolerance specification or better
- Low temperature coefficient

Loading Custom motorBench Firmware Packages in a Project for Advanced Users

Microchip may provide an updated or modified firmware package for MCAF in certain circumstances. To use a custom firmware package in a project, follow these instructions:

1. Locate the project's directory location, which can be found by right clicking on the project name in the Projects tab and navigating to **Properties** → **General** → **Project Location**.
2. Navigate to the project's folder in the system file explorer.
3. Create a "motorbench" folder in the project's folder if one does not exist.
4. Place the custom firmware package as a .zip file in the motorbench folder.
5. If MCC or motorBench are already running, close MCC by clicking on the MCC icon on the toolbar.
6. Launch MCC and motorBench® Development Suite, and there will be red text in the gray margin of motorBench indicating a custom firmware package is in use as seen below.

